

Доклад Модель сегрегации Шеллинга

Математическое моделирование

Боровиков Д.А.

Российский университет дружбы народов им. Патриса Лумумбы, Москва, Россия

- Боровиков Даниил Александрович
- НПИБд-01-22
- Российский университет дружбы народов
- [1132222006@pfur.ru]

В данном докладе мы познакомимся с возможностями агентных моделей, которые используются для понимания сложных явлений. Для этого мы воспользуемся Python и моделью Шеллинга.

— это агентная модель, разработанная экономистом Томасом Шеллингом. Модель Шеллинга не включает внешние факторы, которые оказывают давление на агентов с целью сегрегации, но работа Шеллинга демонстрирует, что наличие людей с “умеренным” внутригрупповым предпочтением по отношению к своей собственной группе все еще может привести к сильно сегрегированному обществу через фактическую сегрегацию.

В агентно-ориентированных моделях есть три параметра: 1) агенты, 2) поведение (правила) и 3) показатели на агрегированном уровне. В модели Шеллинга агентами являются люди, живущие в городе, поведением — переезд в другой район в зависимости от коэффициента сходства, а показателями на агрегированном уровне — коэффициент сходства.

Python-реализация модели Шеллинга

```
class Schelling:
    def __init__(self, width, height, empty_ratio, similarity_threshold, n_iterations, races=2):
        # Инициализация параметров модели
        self.width = width # Ширина сетки
        self.height = height # Высота сетки
        self.races = races # Количество групп (рас)
        self.empty_ratio = empty_ratio # Доля пустых домов
        self.similarity_threshold = similarity_threshold # Порог схожести для удовлетворённости
        self.n_iterations = n_iterations # Максимальное количество итераций
        self.empty_houses = [] # Список пустых домов
        self.agents = {} # Словарь агентов (координаты: группа)
```

Figure 1: Конструктор

Python-реализация модели Шеллинга

```
def is_satisfied(self, x, y):  
    # Проверка, удовлетворён ли агент на позиции (x, y)  
    race = self.agents[(x, y)] # Группа агента  
    count_similar = 0 # Количество соседей той же группы  
    count_different = 0 # Количество соседей другой группы  
  
    # Список координат всех 8 соседей  
    neighbors = [  
        (x - 1, y - 1), (x, y - 1), (x + 1, y - 1),  
        (x - 1, y), (x + 1, y),  
        (x - 1, y + 1), (x, y + 1), (x + 1, y + 1)  
    ]  
  
    for nx, ny in neighbors:  
        # Проверка, находится ли сосед в пределах сетки и не является ли дом пустым  
        if (0 <= nx < self.width and 0 <= ny < self.height and  
            (nx, ny) not in self.empty_houses and (nx, ny) in self.agents):  
            if self.agents[(nx, ny)] == race:  
                count_similar += 1  
            else:  
                count_different += 1  
  
    # Если соседей нет, агент удовлетворён  
    if count_similar + count_different == 0:  
        return False  
    # Агент не удовлетворён, если доля похожих соседей ниже порога  
    return count_similar / (count_similar + count_different) < self.similarity_threshold
```

Reader Mode

Figure 2: Функция удовлетворенности

Python-реализация модели Шеллинга

```
def update(self):
    # Выполнение симуляции для заданного числа итераций
    for i in range(self.n_iterations):
        self.old_agents = copy.deepcopy(self.agents) # Копия текущего состояния
        n_changes = 0 # Счётчик перемещений
        for agent in self.old_agents:
            if self.is_unsatisfied(agent[0], agent[1]):
                agent_race = self.agents[agent] # Группа агента
                empty_house = random.choice(self.empty_houses) # Выбор пустого дома
                self.agents[empty_house] = agent_race # Перемещение агента
                del self.agents[agent] # Удаление старой позиции
                self.empty_houses.remove(empty_house) # Обновление списка пустых домов
                self.empty_houses.append(agent)
                n_changes += 1
        print(f"Итерация {i + 1}: {n_changes} агентов перемещено")
        if n_changes == 0: # Прерывание, если перемещений нет
            break
```

Figure 3: Функция симуляции

Модель Шеллинга с 2 группами: Начальное состояние

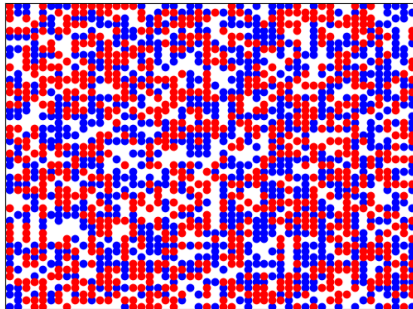


Figure 4: Модель Шеллинга начальное состояние

Модель Шеллинга с 2 группами: Финальное состояние, порог 30%

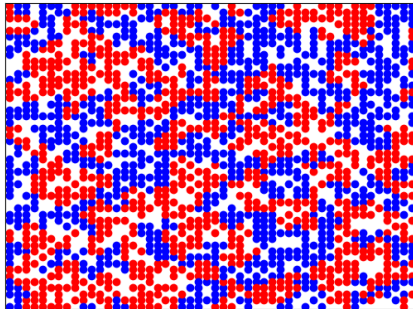


Figure 5: Модель Шеллинга порог сходимости 30%

Модель Шеллинга с 2 группами: Финальное состояние, порог 50%

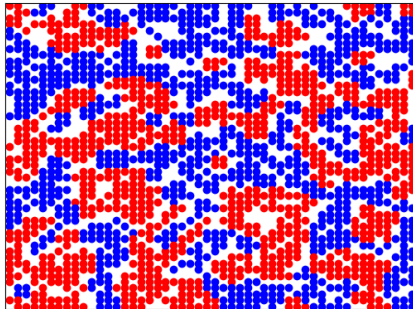


Figure 6: Модель Шеллинга порог сходимости 50%

Модель Шеллинга с 2 группами: Финальное состояние, порог 70%

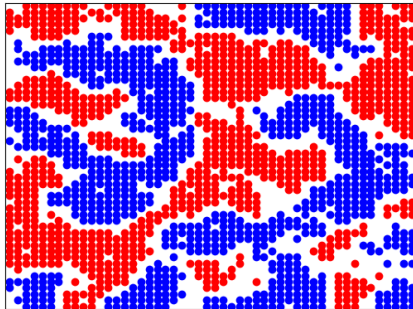


Figure 7: Модель Шеллинга порог сходимости 70%

Порог схожести vs Средний коэффициент схожести

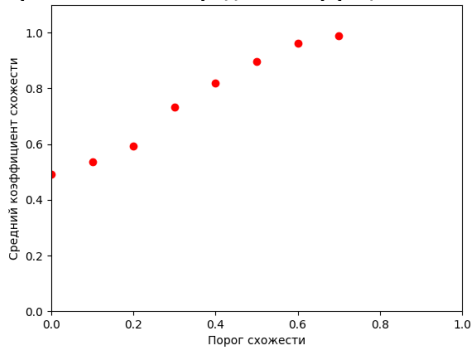


Figure 8: График зависимости среднего значения сходства для разных порогов сходства

Мы рассмотрели один пример агентных моделей под названием «Модель сегрегации Шеллинга» и реализовали её на Python. Эта очень простая модель помогла нам понять очень сложное явление, а именно сегрегацию в многонациональных городах. Мы смогли показать, что очень высокий уровень сегрегации в этих городах не обязательно приводит к нетерпимости на индивидуальном уровне.